24BCE1270
Manan Dudeja

Design And Analysis of Algorithms
Inlab practice 13- Floyd Warshall

- ## Logic-

Floyd-Warshall algorithm finds the shortest paths between every pair of vertices in a weighted graph.

It uses a dynamic programming approach by progressively allowing each vertex to act as an intermediate vertex in the path between two nodes.

Time Complexity- $O(V^3)$

- ## Pseudocode-

Initialize distance matrix using graph weights

```
FOR k = 1 to V
  FOR i = 1 to V
    FOR j = 1 to V
      IF dist[i][j] > dist[i][k] + dist[k][j]
dist[i][j] = dist[i][k] + dist[k][j]
```

- ## Cpp Code:

```
#include <iostream>
#include <vector>

#include <fstream>

#include <ctime>


using namespace std;
```

```
#define INF 99999

void floydWarshall(vector<vector<int>>& dist, int V) {

   for(int k = 0; k < V; k++)

       for(int i = 0; i < V; i++)

           for(int j = 0; j < V; j++)

               if(dist[i][k] + dist[k][j] < dist[i][j])

                   dist[i][j] = dist[i][k] + dist[k][j];

}

int main(int argc, char* argv[]) {

   if(argc < 2){

       cout << "Usage: ./a.out inputfile\n";

       return 0;

   }

   ifstream file(argv[1]);

   if(!file){

       cout << "File not found\n";

       return 0;

   }

   int V;

   file >> V;

   vector<vector<int>> graph(V, vector<int>(V));
```

```
    for(int i=0;i<V;i++)
```

```
        for(int j=0;j<V;j++)
            file >> graph[i][j];


    clock_t start = clock();


    floydWarshall(graph, V);


    clock_t end = clock();


    double timeTaken = double(end - start) / CLOCKS_PER_SEC;


    cout << "Computation finished\n";
    cout << "Time Taken: " << timeTaken << " seconds\n";


    return 0;
}
```

- <u>Python Code:</u>
  - Average case code:

```
import random
sizes = [10, 20, 50, 100]
INF = 99999
for n in sizes:
    filename = "floyd_" + str(n) + ".txt"
    with open(filename, "w") as f:
        f.write(str(n) + "\n")
        for i in range(n):
            row = []
```

```
        for j in range(n):
            if i == j:
                row.append(0)
            else:
                if random.random() < 0.6:
                    row.append(random.randint(1,20))
                else:
                    row.append(INF)
        f.write(" ".join(map(str,row)) + "\n")
print("Generated:", filename)
```

Instructions to check runtime:

1. Create a folder on Desktop.
2. Save:

- C++ file →floyd.cpp
- Python file →case.py

3. Open terminal in the folder. 4. Generate input files:

python3 case.py

5. Compile C++ program:

g++ floyd.cpp 6. Run program:

./a.out floyd_200.txt

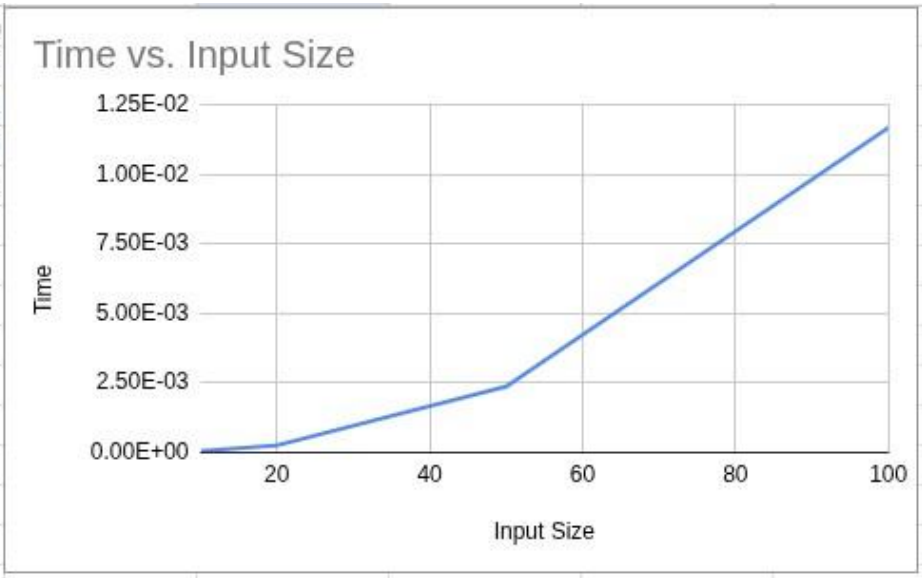
7. Repeat for different input sizes.
8. Note down execution time.
9. Plot Input Size vs Time Taken graph in MS Excel.

Output:

```
student@ab1205b-50:~/Downloads/hello$ g++ floyd.cpp
student@ab1205b-50:~/Downloads/hello$ ./a.out floyd_10.txt
Computation finished
Time Taken: 2.9e-05 seconds
student@ab1205b-50:~/Downloads/hello$ ./a.out floyd_20.txt
Computation finished
Time Taken: 0.000231 seconds
student@ab1205b-50:~/Downloads/hello$ ./a.out floyd_50.txt
Computation finished
Time Taken: 0.002356 seconds
student@ab1205b-50:~/Downloads/hello$ ./a.out floyd_100.txt
Computation finished
Time Taken: 0.011682 seconds
```

Graph-

Vertices	Time (seconds)
10	2.90E-05
20	2.31E-04
50	0.002356
100	0.011682



Result-

Floyd-Warshall algorithm computes shortest paths between all pairs of vertices using dynamic programming.
The algorithm has $O(V^3)$ time complexity, which becomes expensive for large graphs but works well for dense graphs.